cq-006 — Quiz Competition: Qualifiers (ABC 475 E)

Platform	atcoder
Link	https://atcoder.jp/contests/abc475/tasks/abc475_e
Solution files	`e.cpp` · `e2.cpp` · `e3.cpp`
Status	partial
Difficulty	ABC position E (official rating not recorded)
Concepts	binary search with the STL, prefix descent, binary trie, fenwick tree, coordinate compression, offline query processing, bit packing, complexity accounting
Documented	2026-09-18

> **Duplicate — edit both copies.** This is the standalone write-up for one problem of `ct-01`, kept separate from `CONTEST.md` because three solutions and eight concepts do not fit a contest-set table row. A verbatim copy lives at `atcoder/abc475-e-qualifiers.md`, so the problem carries a normal `cq-NNN` row in `INDEX.md` too. The two files differ only in relative link depth.

---

1. Premise

`N` participants each answer `K` two-choice questions. Every answer is `o` or `x`. String `T` holds the correct answers; string `S_i` holds participant `i`'s answers. At most `M` participants advance.

Who advances is decided by a procedure, not by a score. Every participant starts *undetermined*. Walk the questions in order `1 … K`, carrying a running count of confirmed qualifiers:

- Let `A` be the undetermined participants who answered question `k` **correctly**.
- If `qualifiers + |A| ≤ M`, every participant in `A` becomes a **qualifier**. The undetermined who got `k` wrong stay undetermined.
- Otherwise, every undetermined participant who got `k` **wrong** becomes a **non-qualifier**. The rest stay undetermined.

Anyone still undetermined after question `K` is a non-qualifier.

Read that twice. It is not "top `M` by number correct" — earlier questions dominate later ones completely, and a participant can be eliminated while still having more correct answers than someone who advanced.

Queries. `Q` queries, each a pair `(i, j)`: participant `i` flips their answer to question `j`, then you report whether `i` advances. **Flips persist** into every later query, so the state accumulates.

Constraints that matter:

bound	what it signals
$1 \le M \le N \le 3\times10^4$	N alone is small
$1 \le K \le 200$	K is tiny — a per-query factor of K is affordable
$1 \le Q \le 5\times10^4$	$K\cdot Q = 10^7$ is fine; $N\cdot Q \approx 1.5\times10^9$ is not

That last row is the whole constraint story: **re-examining all N participants per query is already too slow, before you have done anything clever with them.**

Output is `Yes` / `No`, capitalised exactly. The statement carries no case-insensitivity note.

Where you got stuck: not attempted during the contest — the link was recorded and the "solution idea" comment left blank. Afterwards you built the right algorithm (sorted array plus a `partition_point` descent) and it ran correct but slow. Your own framing of the blocker: *"e.cpp (my sol) is correct but giving a TLE ... the complexity should be O(QKlogN) which to me seems under the budget."* That reading was right about the descent and wrong about the query as a whole — §2.3 is the concept that closes the gap.

---

2. Concepts you need

Everything below is a general tool, explained without reference to this problem. Read §2, close the file, re-attempt. The problem-specific insight is in §3.

2.1 Fixed-length bitstrings, lexicographic order, and prefix contiguity

What it is: a *bitstring* is a string over `{'0','1'}`. When every bitstring in a collection has the **same length**, lexicographic order (dictionary order, the order `std::string`'s `<` and `std::sort` use by default) coincides with reading the string as a binary numeral: position 0 is the most significant bit.

How it works. Compare two equal-length bitstrings by scanning left to right and stopping at the first position where they differ; whichever has `'1'` there is greater. Every position to the right is irrelevant. So `"1000"` > `"0111"` — one `'1'` in the leading position outweighs any number of later ones.

The property that matters — prefix contiguity. Sort a collection of equal-length bitstrings. Then *for any prefix `P`, the strings beginning with `P` occupy one contiguous run of the sorted array.*

Why: suppose $a \le b \le c$ in sorted order and both a and c start with P (length L). If b did not start with P, then b differs from P at some position $t < L$. At position t, a and c both carry $P[t]$. If $b[t] = '1'$ and $P[t] = '0'$ then $b > c$; if $b[t] = '0'$ and $P[t] = '1'$ then $b < a$. Either way b falls outside $[a, c]$, contradicting its position. So b starts with P. ∎

Contiguity is what lets an **index range** `[lo, hi)` stand in for the *set* "all strings with prefix `P`". Ranges are cheap; sets are not.

Sorting the other way. `std::sort(v.begin(), v.end(), std::greater<std::string>())` gives descending order. Contiguity is unaffected — only the direction changes, so a prefix group is still one run. Descending is the useful direction whenever `'1'` means "better" and you want the best candidates at low indices.

Complexity: comparing two length-`K` strings is **O(K)**, not O(1). Sorting `n` of them is therefore O(n log n · K) character operations. Remember this; §2.3 is built on it.

When to reach for it: any time objects are described by a fixed-length sequence of binary attributes and the attributes have a strict priority order.

Worked micro-example. Six 3-bit strings, sorted descending:

```
index:   0     1     2     3     4     5
value:  111   110   101   100   011   000
```

Prefix `"1"` → range `[0, 4)`. Prefix `"10"` → range `[2, 4)`. Prefix `"100"` → range `[3, 4)`. Each is one run, and each is nested inside the previous.

Common mistakes:

- Assuming `std::string` comparison is O(1). It is O(length), and at `K = 200` that factor is the difference between passing and timing out.
- Sorting descending and then reusing index arithmetic written for ascending order. In descending order the *m*-th largest sits at index `m-1`; in ascending order it sits at `n-m`. Mixing the two produces a mirrored off-by-one that still passes small tests.

Sources:

- `std::sort` — cppreference
- `std::greater` — cppreference
- Competitive Programmer's Handbook (Laaksonen), ch. 3 "Sorting"

2.2 `partition_point` on a sub-range

`ct-01` §2.4 already covers the STL binary-search family in depth — the true-then-false law, `partition_point` as the general form with `lower_bound` and `upper_bound` as special cases, descending ranges needing a comparator, and the random-access requirement. **Read that section first.** Three things are new here.

(a) Searching an interval `[L, R]` is just passing sub-range iterators. There is no separate overload. `partition_point` takes a half-open range, so an *inclusive* `[L, R]` becomes `begin() + L, begin() + R + 1`:

```
auto it  = std::partition_point(v.begin() + L, v.begin() + R + 1, pred);
int  idx = static_cast<int>(it - v.begin());   // first index in [L,R] where pred is false
```

`idx == L` means the predicate was false throughout; `idx == R + 1` means it was true throughout and `it` is the end iterator, so dereferencing it is a bug. `idx - 1` is the last true index and only exists when `idx > L`.

(b) The precondition is local to the sub-range. `partition_point` requires the range it is handed to be partitioned — all true, then all false. It never inspects anything outside `[first, last)`, so **the surrounding array may be completely unpartitioned**. That is exactly what makes the call composable: you can narrow to a sub-range where a predicate happens to be monotone even though it is not monotone globally.

The flip side: when you narrow the range yourself, *you* now owe that precondition. Handing `partition_point` a range that is not partitioned is undefined behaviour, and it does not announce itself — no crash, no warning, just a plausible-looking index. `std::is_partitioned` exists precisely to assert this in a debug build.

© **Predicate cost is the whole complexity story.** `partition_point` performs O(log n) *predicate applications*. If the predicate is `str[d] == '1'` — one character read — the search is O(log n) genuinely. If it is a full string comparison it is O(K log n), and if it is an O(n) feasibility check it is O(n log n). A binary search is only as cheap as one evaluation of its predicate.

Complexity: O(log n) predicate applications on random-access iterators.

When to reach for it: the property you are searching for is monotone only inside some window, or the predicate is a cheap direct test rather than a comparison against a value.

Worked micro-example. Using the six strings from §2.1, restricted to `[2, 4)` (the prefix-`"10"` block), with predicate “bit 2 is `'1'`”:

```
index:    2      3
value:   101    100
pred:    true   false     ->  partition_point returns index 3
```

The same predicate over the whole array reads `true false true false true false` — not partitioned, and the call would be undefined. The block is what makes it legal.

Common mistakes:

- `v.begin() + lo + 1` when `lo` can be `-1`. C++ evaluates `+` left to right, so this computes `v.begin() + (-1)` as an intermediate before adding 1 — a pointer formed outside the array, which is undefined behaviour even though the value is discarded immediately. Write `v.begin() + (lo + 1)`. A build with `-D_GLIBCXX_DEBUG` catches it instantly; an optimised build silently does the right thing until it doesn't.
- Capturing the wrong variable in the predicate lambda. `[k](const std::string& s){ return s[k] == '1'; }` inside a loop over `i` reads position `k`, not `i`. When `k` is the string length this is *not* a crash: `std::string::operator[]` at index `size()` is defined and returns `'\0'`, so the predicate is uniformly false and the search silently returns `first` every time.

Sources:

- `std::partition_point` — cppreference
- `std::is_partitioned` — cppreference
- `std::basic_string::operator[]` — cppreference (the `size()` index returns the null character)
- Operator precedence and associativity — cppreference
- libstdc++ debug mode
- ct-01 §2.4 — binary search with the STL

2.3 Complexity accounting: cost the whole query, not the clever part

What it is: the discipline of writing down *every* per-operation step with its cost, then summing — rather than analysing the algorithmically interesting step and assuming the rest is free.

How it works. For each line executed per query, ask two questions: how many times does it run, and what does one run cost? Setup that happens once is a separate budget from work that repeats. The trap is that the *boring* lines — a container copy, a re-sort, a rebuild — are usually the expensive ones, precisely because they are boring enough not to be counted.

Two costs hide especially well:

- **Copying a container.** `auto S = C;` where `C` is `vector<string>` of `n` strings of length `k` is O(n·k) character copies **plus** `n` **heap allocations**. It looks like one line of bookkeeping.
- **Re-deriving state that barely changed.** Sorting an array that differs from the last sorted version by one element still costs a full sort. The fix is almost always to maintain the structure incrementally, or to build it once outside the loop.

Worked micro-example. Suppose per query you copy an array of `n = 3×10⁴` strings of length `k = 200`, sort it, then run `k` binary searches with an O(1) predicate, over `q = 5×10⁴` queries:

step	per query	× q
copy	`n·k` = 6×10^{6}	3×10^{11}
sort	`n log n · k` = 9×10^{7}	4.5×10^{12}
`k` × binary search	`k log n` = 3×10^{3}	1.5×10^{8}

The step you designed is four orders of magnitude cheaper than the two you did not think about. Both "expensive" rows are removable — the question is never "is my algorithm fast", it is "is every line in the loop fast".

When to reach for it: whenever a solution you believe is within budget times out. Write the table before touching the code.

Common mistakes: quoting a complexity for the interesting subroutine and calling it the complexity of the solution; treating string or container operations as O(1) because they are one token in the source.

Sources:

- Competitive Programmer's Handbook (Laaksonen), ch. 2 "Time complexity"

2.4 Binary trie

What it is: a *trie* (prefix tree) is a tree in which each edge carries one symbol and each node represents the prefix spelled by the path from the root. A **binary trie** is the case where the alphabet is `{0, 1}`, so every node has at most two children. Depth `d` nodes correspond exactly to length-`d` prefixes.

How it works. Store an integer `cnt` on every node: how many stored strings pass through it, i.e. how many have that node's prefix. Then:

- **Insert** a length-`K` string: walk from the root, creating a child when one is missing, incrementing `cnt` on every node along the way. O(K).
- **Erase**: walk the same path decrementing `cnt`. O(K).
- **Count strings with prefix `P`**: walk `P` from the root and read `cnt` at the node you land on. O(|P|), and O(1) if you are already standing on that node.

That last operation is the point. A trie answers "how many strings start with `P`" by *being* the index, with no search.

The relationship to a sorted array. A binary trie and a sorted array of equal-length bitstrings encode the same information. Prefix contiguity (§2.1) says a prefix group is an index range; the trie says a prefix group is a node. Descending a trie one bit at a time and binary-searching a sorted array for successively longer prefixes are the *same traversal* — the trie precomputes the boundaries the binary search would recompute. Trie: O(1) per level, O(N·K) memory. Sorted array: O(log N) per level, O(N) memory. That is the whole trade.

Node storage and the null sentinel. Represent nodes as parallel arrays indexed by integer id — `vector<array<int,2>> ch` and `vector<int> cnt` — rather than pointers. Use id `0` as a **null sentinel**: a node whose count is zero and whose children are both `0`. Walking off the end of the trie then lands on `0` and stays there, reporting empty forever, with no null check anywhere. Make the root id `1`, so that "child is `0`" unambiguously means "absent".

Node recycling. With `Q` erases and inserts, naively allocating a fresh node on every insert grows the arrays without bound. Keep a **free list**: when an erase drops a node's `cnt` to zero, nothing else lives beneath it, so the rest of the subtree is exactly the remainder of the string being erased. Detach it and push every node on that tail onto the free list; `newNode()` pops from the free list before growing. Live nodes then stay bounded by the current contents.

Complexity: O(K) insert, erase, and prefix-count. Memory is at most `N·K + 2` nodes — more precisely `Σ_d min(2^d, N)`, since the first `log₂ N` levels cannot hold more nodes than they have distinct prefixes.

When to reach for it: queries about *prefixes* of fixed-length keys — counting, maximising XOR, k-th element in bit order — especially when the key set changes between queries.

Worked micro-example. Insert `110`, `100`, `101`. Counts per node:

```
          root(3)
         /
       1(3)
      /    \
   0(2)     1(1)
   /  \        \
0(1)  1(1)     0(1)
```

"How many start with `10`?" → walk `1`, `0` → node `0(2)` → answer 2. One lookup, no comparisons.

Common mistakes:

- Using node id `0` as both the root and the null marker, so falling off the trie silently restarts at the root and reports the full set as if it were a sub-prefix.
- Letting the node arrays grow on every insert without a free list, turning `O(N·K)` memory into `O((N + Q)·K)`.
- Assuming a trie is always the fastest option. It is `O(1)` per level but the nodes are scattered in memory; see §3.5 for a measured case where an `O(log N)` array beats it on wall clock.

Sources:

- Trie — Wikipedia
- Aho–Corasick — cp-algorithms (builds a trie from scratch in its first section)
- Bitwise operations — USACO Guide

2.5 Fenwick tree (binary indexed tree)

What it is: an array structure supporting two operations on a sequence of `n` numbers, both in O(log n): **point update** (add `δ` to one position) and **prefix sum** (total of positions `[0, i)`). A range sum `[l, r)` is `prefix(r) - prefix(l)`.

How it works. `tree[i]` stores the sum of a block of the original array ending at `i`, whose length is `i & -i` — the lowest set bit of `i`. Walking `i += i & -i` climbs to the nodes a position contributes to (update); walking `i -= i & -i` walks the disjoint blocks that tile `[0, i)` (query). Both walks take one step per set bit, so O(log n).

Use **1-based** internal indexing — position `0` would make `i & -i` zero and loop forever — while exposing a 0-based interface:

```
void add(int i, int delta) { for (i++; i < (int)tree.size(); i += i & -i) tree[i] += delta; }
int prefix(int i) const { int s = 0; for (; i > 0; i -= i & -i) s += tree[i]; return s; }  // [0, i)
```

Why it, rather than a plain prefix-sum array. A prefix-sum array answers queries in O(1) but costs O(n) to update. When updates and queries interleave — which is what "queries with persistent modifications" means — the O(log n)/O(log n) balance wins.

Complexity: O(log n) per operation, O(n) memory, and a small constant: two integer ops per loop iteration.

When to reach for it: you need running counts or sums over an index range while individual positions keep changing. The canonical framing is *"how many live items are currently in this index range"*.

Worked micro-example. `n = 8`, all zero. `add(3, 1)` then `add(5, 1)`: `prefix(4)` = 1 (only position 3), `prefix(6)` = 2, `prefix(8)` = 2. Then `add(3, -1)`: `prefix(4)` = 0, `prefix(8)` = 1.

Common mistakes:

- Off-by-one between "prefix of `i` elements" and "prefix up to index `i`". Pick one convention and write it in a comment beside the function. This file uses `prefix(i) = sum of [0, i)`, so the count of elements at ranks `≤ r` is `prefix(r + 1)`.
- Forgetting the 1-based shift and looping forever at index 0.

Sources:

- Fenwick tree — cp-algorithms
- Point update, range sum — USACO Guide
- Fenwick tree — Wikipedia

2.6 Coordinate compression

What it is: replacing values from a large or unbounded domain with their **rank** in the sorted list of values that actually occur, so that an array indexed by value becomes affordable.

How it works. Collect every value that will ever appear, sort, remove duplicates, and map each value to its index in that deduplicated list via binary search. A structure indexed by value (a Fenwick tree, a bucket array) then needs only as many slots as there are distinct values, regardless of how large the values are.

```
std::sort(vals.begin(), vals.end());
vals.erase(std::unique(vals.begin(), vals.end()), vals.end());
auto rank = [&](const T& x){ return std::lower_bound(vals.begin(), vals.end(), x) - vals.begin(); };
```

Compression preserves order: `rank(a) < rank(b)` exactly when `a < b`. So anything you could ask about the ordering of values, you can ask about their ranks. The comparator used for sorting and for `lower_bound` must be the same one — mixing ascending sort with a `std::greater` lookup is undefined behaviour, not a wrong answer.

Complexity: O(V log V) to build for `V` values, O(log V) per lookup, O(V) memory.

When to reach for it: the values are huge, sparse, or not integers at all (tuples, strings, 200-bit keys), but the *count* of distinct values is small.

Worked micro-example. Values `{1000000, 5, 1000000, 42}` → sorted-unique `{5, 42, 1000000}` → ranks `{2, 0, 2, 1}`. A Fenwick tree over 3 slots now replaces one over 10^6.

Common mistakes:

- Compressing only the values present at the start when later operations introduce new ones. Every value that will *ever* exist has to be in the table — which is what §2.7 is for.
- Forgetting `std::unique` needs the `erase` to actually shrink the vector; it only moves duplicates to the tail and returns the new logical end.

Sources:

- `std::unique` — cppreference
- `std::lower_bound` — cppreference
- Sorting with custom comparators — USACO Guide

2.7 Offline query processing

What it is: reading *all* queries before answering any, so that answering can use knowledge of what is still to come.

How it works. A problem is **online** if each answer must be produced before the next query is readable — interactive problems, and problems where a query depends on a previous answer.

Everything else is **offline**, and offline is strictly more powerful: you may reorder the queries, sort them, batch them, or — the use here — **precompute the entire future**.

The enabling question is: *what does the sequence of queries determine, before any of them is answered?* When each query's effect on the state is a deterministic function of the input (say, "flip bit `j` of participant `i`"), you can simulate the whole sequence up front and collect every state that will ever exist. That turns an unbounded, evolving value domain into a finite, known list — which §2.6 can then compress.

Complexity: the simulation pass costs whatever one state transition costs, times `Q`. The memory cost is storing the queries plus whatever the pass collects.

When to reach for it: a structure would be easy to build if you knew the final set of keys, and the keys are determined by the input rather than by the answers.

Worked micro-example. Queries `add x`, `add y`, `remove x` over arbitrary 64-bit values. Online, you need a balanced BST or a hash map. Offline, `{x, y}` is known before you start, so a 2-slot array indexed by compressed rank suffices.

Common mistakes:

- Applying it where the problem is genuinely online. Check for an interactive protocol, or for queries whose parameters are derived from earlier answers (a common "forced online" device).
- Simulating the transitions to collect keys, then forgetting to **reset the state** before the real pass.

Sources:

- Competitive Programmer's Handbook (Laaksonen), ch. 27 "Square root algorithms" (Mo's algorithm — the standard offline reordering technique)
- Sqrt decomposition and Mo's algorithm — cp-algorithms

2.8 Bit packing into machine words

What it is: storing a length-`K` bitstring as `[K/64]` unsigned 64-bit integers instead of `K` `char`s, shrinking it by 8× and making comparison a handful of integer operations instead of a loop.

How it works. Put bit `d` in word `d / 64` at position `63 - (d % 64)` — that is, fill each word from its most significant end:

```
using Key = std::array<std::uint64_t, 4>;            // 256 bits, enough for K <= 200

inline int  getBit (const Key& k, int d) { return (k[d >> 6] >> (63 - (d & 63))) & 1ULL; }
inline void flipBit(      Key& k, int d) { k[d >> 6] ^= (1ULL << (63 - (d & 63))); }
```

`d >> 6` is `d / 64` and `d & 63` is `d % 64`, both exact for non-negative `d`.

Why that bit order. Filling from the most significant end makes each word's *numeric* value order agree with the bitstring's *lexicographic* order. `std::array` compares element by element, first

element first — so comparing two `Key`s as arrays gives exactly the lexicographic order of the underlying bitstrings, and `std::greater<Key>` gives the descending order of §2.1. One comparison is at most 4 integer compares instead of up to 200 character compares, and it is branch-predictable and trivially copyable, so containers can `memmove` it.

Bits beyond `K` stay zero in every key, so they never affect an ordering.

Complexity: O(1) bit access, O(K/64) comparison, `8·⌈K/64⌉` bytes per key.

When to reach for it: fixed-length binary keys that are compared or copied often, and any time `K` is large enough that per-character work shows up in a profile.

Worked micro-example. `K = 3`, string `"101"` → word 0 = `1<<63 | 0<<62 | 1<<61` = `0xA000000000000000`. String `"011"` → `0x6000000000000000`. `0xA000… > 0x6000…`, matching `"101" > "011"`. ✓

Common mistakes:

- Filling from the least significant end (`1ULL << (d & 63)`), which reverses the order within each word and silently breaks every comparison.
- Sizing the array from `K` bits without rounding up: `K = 200` needs 4 words, not 3.
- Using `1 << d` rather than `1ULL << d` — a shift past 31 on a 32-bit `int` is undefined behaviour.

Sources:

- `std::array` — cppreference (lexicographic comparison operators)
- Bit manipulation — cp-algorithms
- Bitwise operations — USACO Guide

Re-attempt checkpoint

> You now have every piece. Stop here and try the problem again before reading section 3 — that re-attempt is worth more than the solution is.

---

3. The solution

3.1 The insight

Two observations, in order.

First, only correctness matters. Replace each participant's answer string with a **correctness bitstring** `C[i]`, where `C[i][d] = '1'` iff `S_i[d] == T[d]`. The procedure only ever asks "did this

participant get question `d` right", so `T` and the raw answers can be discarded after this step. A query's flip toggles one bit of `C[i]`, because flipping a two-choice answer flips its correctness.

Second — the real insight — the undetermined set is always a prefix group. Look at what one step does to it:

- On a *qualify* step, the undetermined who got `d` right leave (promoted). The survivors are exactly those with `'0'` at `d`.
- On an *eliminate* step, the undetermined who got `d` wrong leave (rejected). The survivors are exactly those with `'1'` at `d`.

Either way the survivors are the participants matching **one more fixed bit**. By induction, after `d` steps the undetermined set is exactly *the set of participants whose bitstring starts with some specific prefix of length* `d`.

Call the prefix the procedure follows the **boundary path**, and call the descent through it the **prefix descent**. That single structural fact is the whole problem: it converts a procedure that looks like it must touch all `N` participants per step into a walk down `K` nested prefix groups. Everything after this is a question of which data structure answers *"how many participants currently have prefix* `P`*?"* fastest.

Two consequences worth proving, because both solutions rely on them.

Qualifiers are always a rank-prefix. Sort the bitstrings **descending**. Claim: at every step, the qualifiers are ranks `[0, qualified)` and the undetermined block is `[qualified, hi)`. Base case: `qualified = 0`, block `[0, N)`. ✓ On a qualify step, the promoted set is the block's `'1'` side — which, in descending order, is the block's *low-index* part `[qualified, p)` — so the qualifier prefix extends to `[0, p)` and the block becomes `[p, hi)`. ✓ On an eliminate step, the rejected set is the block's `'0'` side `[p, hi)`, the block becomes `[qualified, p)`, and the qualifier prefix is unchanged. ✓ ∎

A tie group never splits. Two participants with identical bitstrings agree on every bit, so at every step they fall on the same side of the split. They are promoted or rejected together, always. ∎ This is why it is safe to ask about a participant's *last* occurrence among equal keys — `upper_bound(...) - 1` — rather than tracking which specific array slot is theirs.

Reading off one participant's verdict. Two equivalent tests, one per solution style:

- **By rank:** participant `i` qualifies iff the number of participants ranked at or above them is `≤ qualified`. This is `e.cpp`'s `it <= upper`.
- **By path:** walk `C[i]` against the boundary path. At the first position `d` where they differ, `path[d] = '0'` means step `d` was a qualify step and `C[i][d] = '1'` puts them in the promoted group → **Yes**; `path[d] = '1'` means step `d` was an eliminate step and `C[i][d] = '0'` puts them in the rejected group → **No**. Never differing means still undetermined at the end → **No**.

3.2 Why the naive approach fails

Simulating the statement directly is O(N·K) per query — scan all participants at each of `K` steps. At `N·Q ≈ 1.5×10⁹` before the `K` factor, that is out by about four orders of magnitude. The constraint table in §1 says so outright; this is a problem where reading the bounds tells you the intended shape before you have an algorithm.

3.3 Approach 1 — sorted array + prefix descent (`e.cpp`)

Sort descending, then run the descent with `partition_point` at each level. The block `[upper+1, lower)` is the undetermined set; `partition_point` with predicate `str[d] == '1'` splits it; `itP + 1` is the count `qualified + |A|` the statement compares against `M`.

```
int lower = n, upper = -1;
for (int d = 0; d < k; d++) {
    auto pred = [d](const std::string& str){ return str[d] == '1'; };
    auto itP = std::partition_point(S.begin() + upper + 1, S.begin() + lower, pred) - S.begin() - 1;
    //                                  ^^^^^^^^^^^ as written in e.cpp; needs
    //                                  parenthesising as (upper + 1) -- see 3.10 pitfall 4
    if (itP < m) upper = itP;     // qualify the '1' side, descend into the '0' side
    else         lower = itP + 1; //  too many: keep only the '1' side
}
return it <= upper;
```

`upper` is the last qualifier's index, so `upper + 1` is `qualified`; the `itP < m` test is precisely `qualified + |A| ≤ M`. The descent is O(K log N) with an O(1) predicate — **this part was always correct and always fast.**

Verified: every configuration for `n ≤ 7, k ≤ 6`, every `M`, every single-bit flip — 8,930,946 evaluations against a direct simulation of the statement, **0 mismatches**.

3.4 Why approach 1 times out

Apply §2.3 to the query body:

line	per query	× Q = 5×10^4
`auto S = C;`	O(N·K) = 6×10^6 char copies + `N` allocations	3×10^{11}
`std::sort(S…, greater)`	O(N log N) compares × O(K) each = 9×10^7	**4.5×10^{12}**
`upper_bound` for `it`	O(log N · K) = 3×10^3	1.5×10^8
`K` × `partition_point`	O(K log N) = 3×10^3	1.5×10^8

Measured: **over 60 s** at maximum constraints, against a 2 s limit.

The `O(QK log N)` estimate was right about rows 3 and 4 and omitted rows 1 and 2. The fix is therefore not to change the descent but to **get the sorted order out of the query loop** — which is what the other two solutions do, in two different ways.

3.5 Approach 2 — binary trie (`e2.cpp`)

Replace the sorted array with the structure that *is* the sorted order (§2.4). A trie node holds `cnt`, so "how many undetermined have prefix `P` + `'1'`" is one lookup instead of a binary search, and the descent becomes O(K) with no search at all.

```
int cur = root, qualified = 0;
for (int d = 0; d < K; d++) {
    int correct = cnt[ch[cur][1]];              // O(1): no search
    if (qualified + correct <= m) { qualified += correct; path[d] = '0'; }
    else                          {               path[d] = '1'; }
    cur = ch[cur][path[d] - '0'];
}
```

A query's flip is `erase(C[i])`, toggle, `insert(C[i])` — O(K) each. The verdict comes from the *by path* test of §3.1. The null sentinel keeps the walk total: when the block empties, `cur` becomes `0`, `cnt[ch[0][1]]` is `0`, `qualified + 0 ≤ M` always holds, and the path takes `'0'` for the remaining levels — correct with no special case.

Complexity: $O((N + Q)\cdot K) \approx 1.6\times10^{7}$. **Memory:** `N·K` nodes ≈ 6×10^{6}, measured at 74 MB.

3.6 Approach 3 — offline + compression + Fenwick (`e3.cpp`)

This one keeps `partition_point` exactly as approach 1 wrote it, and removes the sort by a different route: **sort once, offline.**

The flip sequence is entirely in the input, so §2.7 applies. Simulate the flips up front and collect every key any participant will ever hold — the `N` starting keys plus one per query, at most `N + Q` ≈ 8×10^{4}. Sort *that* once and deduplicate (§2.6). Ranks in this static array never move again, so `partition_point` can binary-search it forever.

What still changes is *which* ranks are occupied, and that is a Fenwick tree (§2.5) of live counts: point update on a flip, prefix sum for "how many live participants in this rank range".

```
int lo = 0, hi = keyCount, qualified = 0, prefLo = 0;
for (int d = 0; d < k; d++) {
    auto pred = [d](const Key& key){ return getBit(key, d) == 1; };
    auto p    = std::partition_point(keys.begin() + lo, keys.begin() + hi, pred) - keys.begin();
    int prefP = live.prefix(p);
    int correct = prefP - prefLo;                       // live count in [lo, p)
    if (qualified + correct <= m) { qualified += correct; lo = p; prefLo = prefP; }
    else                          { hi = p; }
}
```

Three details carry their weight:

- **Keys are bit-packed** (§2.8), so `std::greater<Key>` gives the descending order of §2.1 in at most 4 integer compares, and `getBit` is O(1).
- **`prefLo` carries down the descent.** A range sum needs two prefix queries, but `lo` only ever moves *to* `p`, so the prefix at the new `lo` is the one just computed. That halves the Fenwick work to one query per question.
- **Absent keys in the static array are harmless.** The array contains keys nobody currently holds; they contribute 0 to every Fenwick sum, and they do not disturb the rank range of a prefix group.

Verdict: `live.prefix(qNew + 1) <= qualified` — the *by rank* test of §3.1, which is `e.cpp`'s `it <= upper` unchanged.

Complexity: O((N+Q) log(N+Q)) setup, O(Q·K·log(N+Q)) ≈ 1.7×10^8 queries — the budget originally predicted, now actually met.

3.7 Measured results

Maximum constraints, `N = 3×10⁴`, `M = 1.5×10⁴`, `K = 200`, `Q = 5×10⁴`. Limits 2 s / 1024 MiB.

input	`e.cpp`	`e2.cpp` (trie)	`e3.cpp` (offline+BIT)
all-distinct rows	> 60 s	0.19 s / 74 MB	**0.14 s / 8.3 MB**
`M = 1`	—	0.19 s / 74 MB	0.10 s / 8.3 MB
`M = N`	—	0.19 s / 75 MB	0.14 s / 8.3 MB
all rows identical	—	0.14 s / 23 MB	0.20 s / 8.3 MB
every flip hits question 1	—	—	0.14 s / 8.1 MB

The asymptotically worse solution is the faster one. `e3.cpp` carries an extra `log(N+Q)` factor and still wins on wall clock, because its whole working set is ~2.8 MB of contiguous arrays that stay in cache, while the trie chases pointers through 74 MB. At these sizes the memory hierarchy outweighs a factor of 17 in operation count. This is worth internalising as a default suspicion, not a one-off: **a flat array with a `log` is often faster than a pointer structure without one.**

3.8 Correctness argument

All three rest on §3.1: the undetermined set is a prefix group at every step (induction on the two branch cases), qualifiers are a rank-prefix (induction, proved in §3.1), and tie groups never split (identical keys agree on every bit). Given those, the descent computes exactly the statement's procedure, and both verdict tests read the same partition.

Empirically, against a direct simulation of the statement:

check	result
Official sample	all three ✓
`e.cpp` exhaustive (`n ≤ 7, k ≤ 6`, every `M`, every flip)	8,930,946 evaluations, 0 mismatches
`e2.cpp` exhaustive	2,945,602 evaluations, 0 mismatches
`e3.cpp` exhaustive (whole-file, chained flips)	19,810 inputs, 0 mismatches
Randomised stress, duplicate-heavy and 300-query chains	1,100 cases each, 0 mismatches
ASan + UBSan + `_GLIBCXX_DEBUG`	`e2` / `e3` clean; `e.cpp` flags the iterator issue below

3.9 Walkthrough on the sample

```
5 3 3          N=5 M=3 K=3
oxo            T
oxo oxx xxo xox xoo      S_1 .. S_5
3
5 1 / 1 3 / 4 1          queries
```

Correctness bitstrings: `S_1 = 111`, `S_2 = 110`, `S_3 = 011`, `S_4 = 000`, `S_5 = 001`.

Query 3, `(4, 1)` — the interesting one, since it takes both branches. By this point queries 1 and 2 have already flipped `S_5` to `101` and `S_1` to `110`. Flipping participant 4's question 1 makes `C[4] = 100`. Descending order:

```
rank:    0      1      2      3      4
key:    110    110    101    100    011
```

| `d` | block | `'1'` side | `qualified + |A|` | vs `M=3` | action | path | |—|—|—|—|—|—|—| | 0 | `[0,5)` | `{110,110,101,100}`, 4 | 0 + 4 = 4 | > 3 | eliminate the `'0'` side | `'1'` | | 1 | `[0,4)` | `{110,110}`, 2 | 0 + 2 = 2 | ≤ 3 | qualify them → `qualified = 2` | `'0'` | | 2 | `[2,4)` | `{101}`, 1 | 2 + 1 = 3 | ≤ 3 | qualify → `qualified = 3` | `'0'` |

Boundary path = `100`. Participant 4's key is `100` — it never diverges from the path, so they were still undetermined when the questions ran out → **No**. ✓ Participant 1 (`110`) diverges at `d = 1` where the path is `'0'` and they have `'1'` → **Yes**. Participant 5 (`101`) diverges at `d = 2`, same reasoning → **Yes**.

Expected output `Yes / Yes / No`. ✓

3.10 Pitfalls this problem actually produced

Each of these was hit for real while building `e.cpp`, and each is silent — none crashes, none warns.

1. **Lambda captured the wrong variable.** `[k](const std::string& s){ return s[k] == '1'; }` inside a loop over `i`, where `k` is the string length. Not a crash: `std::string::operator[]` at index `size()` returns `'\0'`, so the predicate was uniformly false and `partition_point` returned `first` every time. Fix: capture and index `i`.
2. **Off-by-one that broke the `partition_point` precondition.** `itP` was the *last* `'1'` index, so the `'0'` block starts at `itP + 1`. Writing `upper = itP` left one `'1'`-string glued to the front of the `'0'` block, destroying the shared-prefix invariant the *next* level depends on (§2.2b). The following level then binary-searched an unpartitioned range and returned an index from a region already eliminated.
3. **Ascending-order arithmetic against a descending sort.** The threshold was written as `itP > n-1-m` with the branches arranged for ascending order. With `std::greater` the *m*-th largest sits at index `m-1`, not `n-m`, so both the landmark and the branch direction were mirrored (§2.1).
4. **Iterator arithmetic associativity.** `S.begin() + upper + 1` with `upper = -1` forms `S.begin() - 1` as an intermediate — undefined behaviour, caught by `-D_GLIBCXX_DEBUG` on every stress seed. Fix: `S.begin() + (upper + 1)`.
5. **Output case.** `yes` / `no` instead of `Yes` / `No`. The statement carries no case-insensitivity note, so this fails every test — including the samples, which is worth knowing if local eyeballing suggested otherwise.
6. **State corruption across queries** (an earlier draft): sorting `C` in place meant `C[i]` stopped referring to participant `i` after the first query. Two *identical* queries returned different answers. Fixed by sorting a copy.

3.11 Edge cases handled

`N = 1`; `M = N` (everyone qualifies at the first step); `M = 1`; all bitstrings identical (one tie group of size `N`); a participant flipping the same bit repeatedly (state must toggle back and forth); the undetermined block emptying before question `K` (handled by the null sentinel in `e2.cpp`, and by an empty rank range contributing 0 in `e3.cpp`); the `'1'` side of a block being empty, which makes `|A| = 0` and always takes the qualify branch harmlessly.

---

4. Takeaways

- **"The undetermined set is always one prefix group" is the transferable shape.** When a procedure repeatedly partitions a set by one binary attribute and discards one side, the surviving set is a prefix group — so a sorted array, or a trie, replaces re-scanning. The signal in a statement is *an iterative elimination whose test at step* `d` *depends only on attribute* `d`.
- **Cost every line in the query loop, not the interesting one.** A correct `O(QK log N)` analysis of the descent sat next to an uncounted `O(N log N · K)` sort in the same loop. When a solution you believe is in budget times out, write the table (§2.3) before touching the code.
- **A binary search's complexity is its predicate's complexity × log n.** `str[d] == '1'` is O(1); a string comparison is O(K). Same call, two different complexities.
- **`partition_point`'s precondition is local, which is both the feature and the trap.** It licenses searching a sub-range of an otherwise unpartitioned array — and it makes maintaining that sub-range's invariant your job, with silent wrong answers as the failure mode rather than a crash.
- **Offline is a resource.** If the queries determine every future state, you can build the structure for all of them before answering the first.
- **Prefer the flat array with a `log` over the pointer structure without one** until measurement says otherwise. Here it was 9× less memory and faster wall clock despite 17× more operations.

5. Similar problems

Already in `context/problem-bank.md`:

- Implement Trie (Prefix Tree) — `lc150-0098`. The structure of §2.4 with no counts; build this first if the trie was the unfamiliar piece.
- Design Add and Search Words Data Structure — `lc150-0099`. Adds wildcard descent, so the walk branches instead of following one path.
- Word Search II — `lc150-0100`. Trie plus backtracking; exercises node pruning, the cousin of §2.4's free list.
- Flowers — `dp-q`. Fenwick tree (§2.5) as a running-maximum index inside a DP, the other standard use of the structure.

6. Related

- ct-01 CONTEST.md — the contest set this problem belongs to.
- ct-01 §2.4 — binary search with the STL — the `partition_point` fundamentals §2.2 builds on.
- `concepts/2026-08-04-amortized-analysis.md` — the free-list recycling in §2.4 keeps trie memory bounded by the same amortized argument as vector growth.
- Editorials for ABC 475 — the official write-ups.